



دانشکده علوم ریاضی
دانشگاه فردوسی مشهد

پروژه دوم درس طراحی الگوریتم

یافتن بهینه‌ترین مجموعه کابل‌های پشتیبان

استاد درس: دکتر رضا قنبری

اعضای گروه:

سارا مسگر مقدم، اسما موحدی تبار

فهرست مطالب

۱	عدم بهینگی راهبرد حریصانه	۲
۱.۱	عدم تولید کم‌هزینه‌ترین زیرگراف دو-یال-همبند	۲
۲.۱	عدم تولید کم‌هزینه‌ترین روش تبدیل MST	۳
۳.۱	نتیجه‌گیری از بخش حریصانه	۳
۲	الگوریتم شاخه و کران (Bound and Branch)	۳
۱.۲	ساختار درخت جستجو و وضعیت گره‌ها	۴
۲.۲	کران پایین (Bound Lower)	۴
۳.۲	کران بالا (Bound Upper)	۵
۴.۲	ترتیب پردازش کابل‌ها (Order Branching)	۶
۵.۲	شرایط هرس (Pruning)	۶
۳	مقایسه خروجی فاز دوم و فاز سوم	۷
۴	نتیجه‌گیری	۷

۱ عدم بهینگی راهبرد حریصانه

راهبرد حریصانه در هر مرحله کابلی را انتخاب می‌کند که بهترین معیار محلی (در این پروژه نسبت هزینه به تعداد یال‌های جدید پوشش داده شده) را داشته باشد. این تصمیم‌ها مستقل از تصمیم‌های آینده گرفته می‌شوند؛ بنابراین تضمینی وجود ندارد که مجموع این انتخاب‌ها بهترین جواب نهایی را تولید کند.

در ادامه دو مورد خواسته شده در صورت پروژه به طور دقیق بررسی می‌شوند.

۱.۱ عدم تولید کم‌هزینه‌ترین زیرگراف دو-یال-همبند

برای نشان دادن عدم بهینگی، مثال نقض زیر را در نظر بگیرید. مجموعه یال‌های درخت پوشای کمینه را به صورت

$$U = \{1, 2, 3, 4, 5, 6\}$$

فرض کنید و کابل‌های پشتیبان (زیرمجموعه‌ها) به شرح زیر هستند:

کابل	هزینه	یال‌های پوشش داده شده
S_1	۱۰	$\{1, 2, 3, 4, 5\}$
S_2	۴	$\{1, 2\}$
S_3	۴	$\{3, 4\}$
S_4	۴	$\{5, 6\}$
S_5	۳	$\{6\}$

الگوریتم حریصانه بر اساس نسبت $\frac{\text{هزینه}}{\text{تعداد یال‌های جدید پوشش داده شده}}$ عمل می‌کند:

۱. مرحله اول:

• S_1 : نسبت $10/5 = 2$

• S_2 : نسبت $4/2 = 2$

• S_3 : نسبت $4/2 = 2$

• S_4 : نسبت $4/2 = 2$

• S_5 : نسبت $3/1 = 3$

بین S_1 تا S_4 که نسبت یکسان دارند، الگوریتم S_1 را انتخاب می‌کند (چون اولین کابل با این نسبت در لیست است).

۲. هزینه فعلی = ۱۰، یال‌های پوشش نشده: {6}

۳. مرحله دوم: تنها کابل S_5 با نسبت $3/1 = 3$ انتخاب می‌شود.

۴. هزینه نهایی = $10 + 3 = 13$

در حالی که جواب بهینه، انتخاب S_2, S_3 و S_4 با هزینه‌ی

$$4 + 4 + 4 = 12$$

است که تمام یال‌ها را پوشش می‌دهد و هزینه‌ی کمتری دارد. بنابراین الگوریتم حریصانه کم‌هزینه‌ترین زیرگراف دو-یال-همبند را تولید نمی‌کند. ■

۲.۱ عدم تولید کم‌هزینه‌ترین روش تبدیل MST

همان مثال نقض بالا نشان می‌دهد که الگوریتم حریصانه در تبدیل درخت پوشای کمینه به یک زیرگراف دو-یال-همبند نیز کم‌هزینه‌ترین روش را پیدا نمی‌کند. زیرا:

• روش حریصانه: انتخاب S_1 و S_5 با هزینه‌ی ۱۳

• روش بهینه: انتخاب S_2, S_3 و S_4 با هزینه‌ی ۱۲

بنابراین راهبرد حریصانه نمی‌تواند بهترین افزودنی (Augmentation) را برای مقاوم‌سازی شبکه بیابد. ■

۳.۱ نتیجه‌گیری از بخش حریصانه

مثال فوق به وضوح نشان می‌دهد که تصمیم‌گیری صرفاً بر اساس نسبت هزینه به پوشش محلی، ممکن است الگوریتم را به سمت انتخاب کابل‌های پرهزینه‌تر سوق دهد، در حالی که ترکیبی از کابل‌های ارزان‌تر می‌تواند همه‌ی یال‌ها را با هزینه‌ی کمتری پوشش دهد. در نتیجه، هیچ تضمینی برای بهینگی الگوریتم حریصانه وجود ندارد و برای یافتن جواب بهینه باید از روش‌های دقیق‌تری مانند شاخه و حد استفاده کرد.

۲ الگوریتم شاخه و کران (Bound and Branch)

برای یافتن جواب بهینه، از روش شاخه و کران استفاده شده است. این روش تمام فضای حالت را به صورت سیستماتیک جستجو می‌کند، اما با استفاده از کران‌های پایین و بالا، شاخه‌های غیرامیدوارکننده را هرس می‌کند.

۱.۲ ساختار درخت جستجو و وضعیت گره‌ها

هر گره در درخت جستجو، یک وضعیت (State) را نشان می‌دهد که شامل اطلاعات زیر است:

- index: اندیس کابل بعدی که باید درباره‌ی آن تصمیم‌گیری شود.
- cost: مجموع هزینه کابل‌های انتخاب‌شده تا این گره.
- covered: یک آرایه بولی به طول $n - 1$ که مشخص می‌کند کدام یال‌های MST پوشش داده شده‌اند.
- selected: لیست کابل‌های انتخاب‌شده در این مسیر.

ریشه درخت: در ابتدا هیچ کابلی انتخاب نشده، هزینه صفر، همه‌ی یال‌ها پوشش نشده و اندیس صفر است.

نحوه انشعاب (Branching): در هر گره، برای کابل فعلی ($\text{backups.get(index)}$) دو شاخه ایجاد می‌شود:

۱. شاخه انتخاب (Take): کابل فعلی انتخاب می‌شود. هزینه به‌روز می‌شود، یال‌های پوشش داده‌شده علامت‌گذاری می‌شوند و کابل به لیست انتخاب‌شده‌ها اضافه می‌شود. سپس index یک واحد افزایش می‌یابد.

۲. شاخه رد (Skip): کابل فعلی رد می‌شود. تنها index یک واحد افزایش می‌یابد.

این فرآیند به صورت بازگشتی (عمق‌اول) ادامه می‌یابد تا زمانی که:

- همه‌ی یال‌ها پوشش داده شوند (جواب کامل).
- یا شاخه هرس شود.
- یا همه‌ی کابل‌ها بررسی شوند.

۲.۲ کران پایین (Bound Lower)

طراحی یک کران پایین معتبر، مهم‌ترین بخش الگوریتم شاخه و کران است. در ابتدا، کران پایینی مبتنی بر جمع کمترین هزینه‌های تکی برای هر یال پوشش نشده (minEdgeCost) در نظر گرفته شد:

$$LB_1(\text{state}) = \text{state.cost} + \sum_{i: \text{uncovered}} \min_{b \text{ covers } i} \text{cost}(b) \quad (1)$$

اما پس از تحلیل دقیق، مشخص شد که این کران معتبر نیست، زیرا ممکن است یک کابل با هزینه‌ی کم، چندین یال را هم‌زمان پوشش دهد و هزینه‌ی واقعی جواب بهینه بسیار کمتر از این جمع باشد. در نتیجه، این کران می‌توانست جواب بهینه را هرس کند. بنابراین این روش کنار گذاشته شد.

در گام بعد، کران پایین ساده‌تری بررسی شد که تنها ارزان‌ترین کابل پوشش‌دهنده‌ی حداقل یک یال پوشش‌نشده را در نظر می‌گرفت:

(۲)

$$LB_2(state) = state.cost + \min_{i \geq state.index} \{ cost(backup_i) \mid backup_i \text{ edge uncovered one least at covers } \}$$

این کران معتبر بود، اما بیش از حد ضعیف عمل می‌کرد، زیرا تنها یک یال (معمولاً ارزان‌ترین) را در نظر می‌گرفت و بقیه‌ی یال‌های پوشش‌نشده را نادیده می‌گرفت. برای تقویت کران پایین و افزایش قدرت هرس، از فرمول نهایی زیر استفاده شد:

$$LB(state) = state.cost + \max_{i: \text{uncovered}} \left(\min_{b \text{ covers } i} cost(b) \right) \quad (۳)$$

به عبارت دیگر، برای هر یال پوشش‌نشده، ارزان‌ترین کابلی که آن را می‌پوشاند محاسبه می‌شود ($minEdgeCost[i]$)، و سپس بیشینه‌ی این مقادیر به عنوان کران پایین در نظر گرفته می‌شود. این مقادیر برای جلوگیری از محاسبه‌ی مکرر، یک‌بار در ابتدای اجرای الگوریتم پیش‌محاسبه و در آرایه‌ی $minEdgeCost$ ذخیره می‌شوند.

اثبات اعتبار: فرض کنید S یک جواب کامل دلخواه از وضعیت فعلی باشد. یال e^* که بیشترین $minEdgeCost$ را دارد، باید در S توسط یکی از کابل‌های باقی‌مانده پوشش داده شود؛ ارزان‌ترین گزینه برای این کار دقیقاً $minEdgeCost[e^*]$ است. بنابراین هزینه‌ی S حداقل $state.cost + minEdgeCost[e^*] = LB(state)$ خواهد بود. پس $LB(state) \leq cost(S)$ برای هر جواب کامل S ، و در نتیجه $LB(state)$ هرگز جواب بهینه را هرس نمی‌کند. $\square$

این کران پایین نسبت به نسخه‌ی LB_2 ، دقیق‌تر است زیرا همه‌ی یال‌های پوشش‌نشده را به‌طور هم‌زمان در نظر می‌گیرد، نه فقط یکی؛ و در نتیجه قدرت هرس بیشتری در عمل ارائه می‌دهد، بدون آنکه اعتبار (Admissibility) آن نقض شود.

۳.۲ کران بالا (Bound Upper)

برای بهبود کارایی، یک جواب اولیه با استفاده از الگوریتم حریصانه (همان روش فاز دوم) ساخته می‌شود و به عنوان $bestCost$ (بهترین جواب یافت‌شده تا کنون) در نظر گرفته می‌شود. این کار باعث می‌شود هرس‌ها زودتر انجام

شوند.

۴.۲ ترتیب پردازش کابل‌ها (Order Branching)

علاوه بر کران پایین و کران بالا، ترتیب پردازش کابل‌های پشتیبان در درخت جستجو نیز بر کارایی الگوریتم شاخه و کران تأثیرگذار است. پیش از شروع جستجو، کابل‌های پشتیبان بر اساس نسبت هزینه به تعداد یال‌های پوشش داده‌شده (به‌طور صعودی) مرتب می‌شوند:

$$score(backup) = \frac{cost(backup)}{|covers(backup)|} \quad (۴)$$

به این ترتیب، کابل‌هایی که نسبت به هزینه‌شان یال‌های بیشتری را پوشش می‌دهند (یعنی از نظر اقتصادی کارآمدترند)، زودتر در درخت جستجو بررسی می‌شوند.

دلیل اهمیت این ترتیب آن است که در جستجوی عمق‌اول، هرچه زودتر به یک جواب کامل و کم‌هزینه برسیم، مقدار bestCost سریع‌تر کاهش می‌یابد. چون شرط هرس (بخش ۵.۲) مستقیماً به bestCost وابسته است، کاهش زود هنگام آن باعث می‌شود شاخه‌های نامناسب زودتر تشخیص داده شده و از فضای جستجو حذف شوند.

لازم به ذکر است که این بهینه‌سازی، مستقل از کران پایین و کران بالا عمل می‌کند و صرفاً بر ترتیب کاوش گره‌ها تأثیر می‌گذارد، نه بر درستی یا اعتبار جواب نهایی؛ الگوریتم صرف نظر از این ترتیب، در نهایت به همان جواب بهینه می‌رسد، اما این مرتب‌سازی سرعت رسیدن به آن را در عمل به‌طور محسوسی افزایش می‌دهد.

۵.۲ شرایط هرس (Pruning)

در هر گره، قبل از انشعاب، سه نوع هرس اعمال می‌شود:

۱. هرس هزینه: اگر $state.cost \geq bestCost$ ، این شاخه نمی‌تواند جواب بهتری از جواب فعلی داشته باشد، پس هرس می‌شود.
۲. هرس امکان‌پذیری: اگر با کابل‌های باقی‌مانده نتوان همه‌ی یال‌ها را پوشش داد (بررسی با تابع `canStillCover`)، شاخه هرس می‌شود.
۳. هرس کران پایین: اگر $lowerBound(state) \geq bestCost$ ، حتی بهترین حالت ممکن از این گره نیز از جواب فعلی بدتر است، پس هرس می‌شود.

ترکیب این سه معیار، فضای جستجو را به شدت کاهش می‌دهد و الگوریتم را برای ورودی‌های متوسط کارآمد می‌سازد.

۳ مقایسه خروجی فاز دوم و فاز سوم

برای مثال نقض ارائه‌شده در بخش ۲، خروجی دو فاز به صورت زیر است:

فاز سوم (Branch and Bound)	فاز دوم (حریصانه)	
S_2, S_3, S_4	S_1, S_5	کابل‌های انتخاب‌شده
۱۲	۱۳	هزینه کل
□ بهینه	□ غیر بهینه	بهینگی

این جدول به وضوح نشان می‌دهد که الگوریتم حریصانه جواب بهینه را از دست می‌دهد، در حالی که روش شاخه و کران با هرس‌های مناسب به جواب بهینه می‌رسد.

۴ نتیجه‌گیری

در این گزارش، ابتدا با ارائه یک مثال نقض دقیق، نشان دادیم که الگوریتم حریصانه فاز دوم:

۱. الزاماً کم‌هزینه‌ترین زیرگراف دو-یال-همبند را تولید نمی‌کند.

۲. الزاماً کم‌هزینه‌ترین روش برای تبدیل MST به زیرگراف دو-یال-همبند را تولید نمی‌کند.

سپس الگوریتم شاخه و کران را به عنوان یک روش دقیق معرفی کرده و ساختار درخت جستجو، نحوه انشعاب و نمایش وضعیت هر گره را به طور کامل تشریح نمودیم. همچنین یک کران پایین معتبر با اثبات ریاضی ارائه گردید و اشاره شد که در طراحی اولیه، کران پایین دیگری (جمع کمترین هزینه‌های تکی) بررسی شد که به دلیل نامعتبر بودن کنار گذاشته شد. علاوه بر این، پس از رد کران جمعی، نسخه‌ای ساده اما معتبر (مبتنی بر ارزان‌ترین کابل پوشش‌دهنده‌ی حداقل یک یال) پیاده‌سازی شد که بعداً با نسخه‌ی دقیق‌تری مبتنی بر بیشینه‌ی کمینه‌هزینه‌ی هر یال پوشش‌نشده جایگزین گردید. این ارتقا بدون تأثیر بر اعتبار یا بهینگی جواب نهایی، قدرت هرس الگوریتم را افزایش داد که خود نشان‌دهنده‌ی فرآیند تکرارشونده‌ی بهبود الگوریتم است. این نشان‌دهنده‌ی تحلیل عمیق مسئله است.

در نهایت، با مقایسه خروجی فاز دوم و سوم در یک مثال مشخص، برتری الگوریتم شاخه و کران از نظر بهینگی تأیید شد. الگوریتم ارائه‌شده با وجود پیچیدگی نمایی در بدترین حالت، با استفاده از هرس‌های مناسب، در عمل عملکرد قابل قبولی دارد و همواره جواب بهینه را تضمین می‌کند.